

Solution Proposal

Topic	Cascading Failure
Prepared by	Ronald Thian
Prepared on	12 September 2025
Approved by	Kar Wai Cheng
Approved on	17 September 2025

Context

On 9th September 2025, an incident [NLINCIDENT-261](#), occurred where the National Lottery (NL) system experienced an outage resulting in the failure to use the Admin Backoffice for data querying and Agent Application for ticket purchase. This disrupted all business operations as all endpoints were unresponsive.

Problem Evaluation

Investigations show that the system experienced a cascading failure when a large queue of long-running database queries were triggered at burst, causing the application to exhaust all of its available database connection pool.

Once all connections within HikariCP connection pool is held by a long-running query, and when new incoming requests cannot borrow a connection within the configured timeout (currently set at default, 30s), HikariCP threw an error back to the application:

None

```
org.springframework.jdbcCannotGetJdbcConnectionException
```

Eventually, this exhaustion of the database connection pool resulted in the cascading failure where it snowballed into a full system outage.

Temporary Preventive Measures and Hotfix

Source Identification and Remediation

Prior to the incident, the team was informed of several spikes in database utilization. The source was identified to be from an internal DFNN staff running bots to perform large amounts of data querying. As this was not a malicious actor, the team did not block the source of the request, but communicated directly the cause and consequences to the involved party.

Hotfix

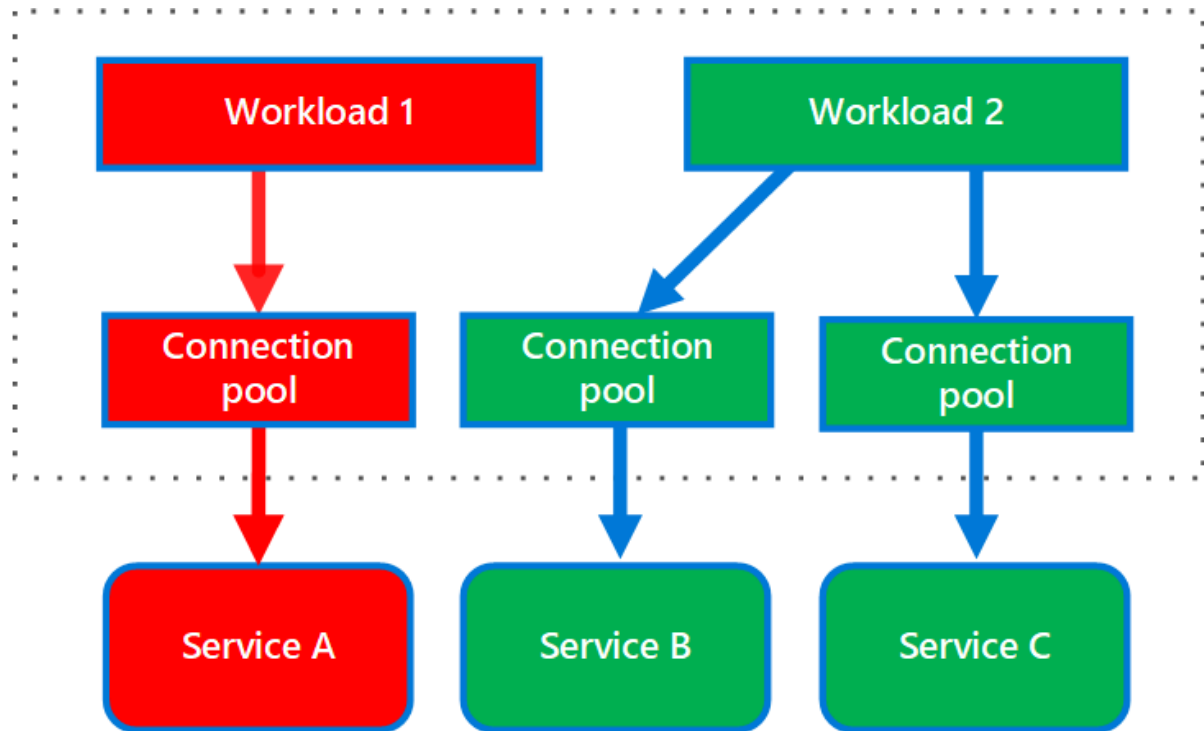
Upon the occurrence of the incident, our developers added a query optimization by adding index onto `bet_group_id`. This changed the query from a full table scan into a more optimized index scan, dropping the query time significantly to allow more headroom for the database connection pool limit.

Proposed Solution

As hotfix only mitigated the likelihood of that particular endpoint, the risk is still wide open which could result in an identical system outage should a similar burst of queries be fired. Hence, there is a need to implement additional preventive measures to ensure system reliability.

Solution 1 - Bulkhead pattern for Connection Pool

In a bulkhead architecture, also known as cell-based architecture, each group of services will have its own dedicated connection pool. In an event of a failure, the other functions can continue operations as per normal. [1]



Option A: Dedicated Connection Pool

As the entire application shares a single HikariCP connection pool, this results in a single point of failure. Adding additional connection pools for different functions within the system would allow isolation of said functions, preventing a full system outage during the exhaustion of all available connection pools.

Option B: Reserved Connections within Connection Pool

In the event where creating an additional connection pool is not feasible, a workaround using application logic can also achieve similar results.

```
Java
// pseudocode

@GetMapping("/reports/some-report")
public ResponseEntity<?> getReport() {
    if(pool.getActiveConnections() > pool.getTotalConnections() / 2) {
        return ResponseEntity.status(HttpStatus.TOO_MANY_REQUESTS)
            .body("Service busy, please try again later!");
    }

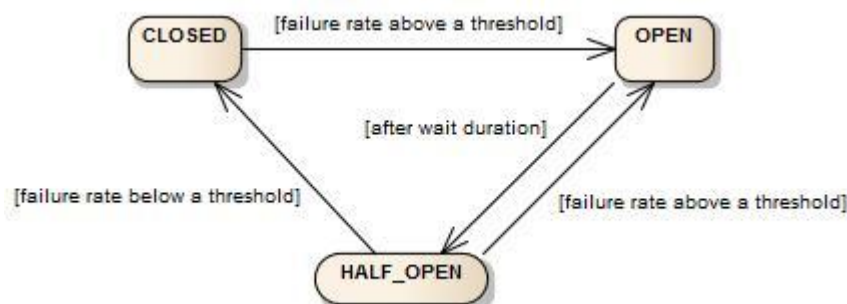
    // resume operations
    // ...
}
```

This would limit the long-running queries to be able to only use a maximum of half of all available connections. However, there is a downside to this as the system is still dependent on a single connection pool.

Solution 2 - Circuit Breaker Pattern

Circuit Breaker Patterns implements a fail-fast methodology into our system, by preventing a retry of a particular call when it has previously timed out or failed. This prevents the rapid build up of the request queue by killing incoming requests until the microservice is ready again. [2] This can be implemented via a finite state machine with three normal states: CLOSED, OPEN, and HALF_OPEN and three special states METRICS_ONLY, DISABLED and FORCED_OPEN [3]

Circuit Breaker Patterns observe incoming requests using different methods during a CLOSED state. In an event of a high rate of failure or timeouts within the incoming requests, the circuit breaker jumps and sets the state into OPEN, hence breaking the circuit and blocks all incoming request from reaching the endpoint. At this point, users will be able to see an error of 429 TOO MANY REQUESTS. Upon a set amount of “cooldown” time, the system toggles into HALF_OPEN, to allow a small amount of request to pass, while monitoring the success rates. Here, the system makes a decision based on the configured failure rate, whether to resume operations by setting it to CLOSED, or to revert into blocking requests via OPEN.



Consideration A: Count-based vs Time-based sliding window

A count-based sliding window is implemented by observing the previous number of requests based on the configured count. It is a simple and responsive setup that fails immediately whenever the incoming requests timeout or fail above the set count. This is excellent for endpoints that have a consistently high amount of traffic.

Contrary to a set count of incoming requests, the time-based sliding window observes incoming requests in a configured time window. This allows a more timely evaluation of the current state of requests as compared to a count based window as it will always observe the newest requests.

Consideration B: Failure rate vs Slow call rate thresholds

The circuit breaker can be set to observe the number of failure rates where an error is thrown, or a configured response time threshold. This allows the ability to not only trigger a circuit breaker measure onto endpoints only upon error throwing, but also during long-running database queries.

Recommendation

It is recommended that both solutions be implemented as they compliment each other in the prevention of a system outage during bursts of requests.

Recommended Solution - Bulkhead Pattern, Circuit Breaker Patten

A bulkhead pattern of **(Option A) creating additional connection pools** dedicated to different groups of endpoints is highly recommended. This prevents the system from a full on outage and allows important functions to continue to run despite high ongoing backoffice queries. Additionally, it would be greatly beneficial to leverage on a circuit breaker library like [resilience4j](#), to quickly implement circuit breaking into our endpoints. It is also recommended to configure it using a **(Consideration A) Time-based sliding window** with **(Consideration B) slow call rate threshold**, as it is better suited for endpoints with large bursts. Call Rate Threshold is also excellent for accurately measuring the database query health, allowing report endpoints to fail-fast early, and allowing the system to naturally recover.

Sources

1. Microsoft. (n.d.). *Bulkhead pattern*. In *Cloud design patterns* (Azure Architecture Center). Retrieved 12 September 2025 from <https://learn.microsoft.com/en-us/azure/architecture/patterns/bulkhead>
2. Amazon Web Services. (n.d.). *Circuit breaker pattern*. In *AWS Prescriptive Guidance: Cloud design patterns, architectures, and implementations*. Retrieved 12 September 2025, from <https://docs.aws.amazon.com/prescriptive-guidance/latest/cloud-design-patterns/circuit-breaker.html>
3. Resilience4j. (n.d.). *CircuitBreaker*. In *Resilience4j — Guides*. Retrieved 12 September 2025, from <https://resilience4j.readme.io/docs/circuitbreaker>